

🕒 Cập nhật tháng 8 năm 2024

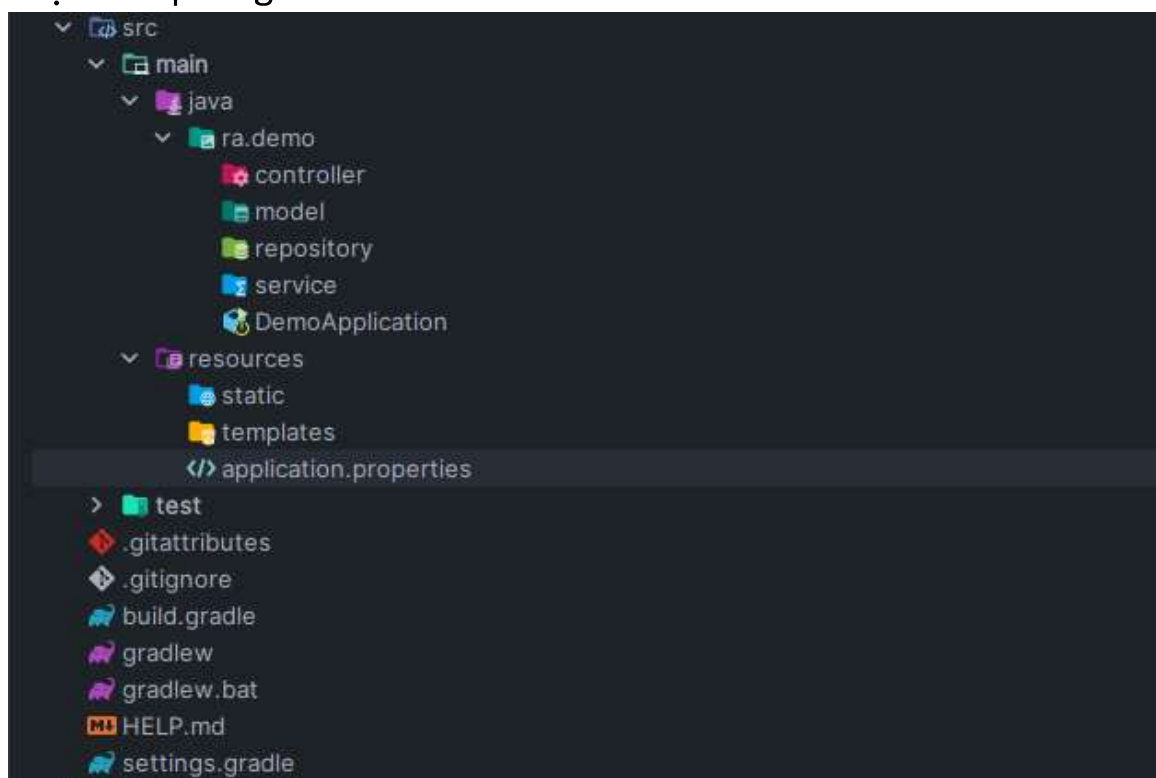
[Bài đọc] Cấu trúc dự án và hướng dẫn tạo dự án dịch vụ web

1. Giới thiệu về dịch vụ web

- **Dịch vụ web (Web Service)** là một ứng dụng phần mềm hỗ trợ giao tiếp qua **giao thức HTTP** giữa các hệ thống khác nhau
- Mục đích: Cung cấp chức năng (API) để hệ thống khác có thể gọi và sử dụng
- Có 2 loại phổ biến:
 - **SOAP Web Service** (XML-based, nặng hơn, ít dùng trong dự án mới)
 - **RESTful Web Service** (sử dụng JSON/XML, gọn nhẹ, phổ biến nhất hiện nay)
- Trong bài này mình tập trung vào **RESTful Web Service** với **Spring Boot + Java**

2. Cấu trúc dự án dịch vụ web (Spring Boot REST API)

- Một dự án Spring Boot cơ bản có thể tổ chức như sau:



- Ý nghĩa:
 - **controller**: nhận request từ client, trả response
 - **service**: xử lý logic nghiệp vụ, có thể gọi nhiều repository
 - **repository**: giao tiếp với database
 - **model**: định nghĩa các entity tương ứng với bảng trong CSDL
 - **resources/application.properties**: cấu hình app (ví dụ: DB URL, user/pass, server.port=8080)

3. Các bước tạo dự án dịch vụ web

- Bước 1: Tạo dự án
 - [Link hướng dẫn tạo dự án](#)
- Bước 2: Tạo Entity (Model)

```
package ra.demo.model;

import jakarta.persistence.*;

@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;

    // getters và setters
}
```

- Bước 3: Tạo Repository

```
package ra.demo.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import ra.demo.model.User;

public interface UserRepository extends JpaRepository<User, Long> {

}
```

- Bước 4: Tạo Service

```
package ra.demo.service;

import org.springframework.stereotype.Service;
import ra.demo.model.User;
import ra.demo.repository.UserRepository;

import java.util.List;

@Service
public class UserService {
    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public List<User> findAll() {
        return userRepository.findAll();
    }

    public User save(User user) {
        return userRepository.save(user);
    }
}
```

- Bước 5: Tạo Controller

```

package ra.demo.controller;

import org.springframework.web.bind.annotation.*;
import ra.demo.model.User;
import ra.demo.service.UserService;

import java.util.List;

@RestController
@RequestMapping("/api/users")
public class UserController {
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    // GET all users
    @GetMapping
    public List<User> getAllUsers() {
        return userService.findAll();
    }

    // POST add new user
    @PostMapping
    public User addUser(@RequestBody User user) {
        return userService.save(user);
    }
}

```

- Bước 6: Cấu hình application.properties

```

1  spring.application.name=demo
2
3  # server port
4  server.port=8080
5
6  # H2 database
7  spring.datasource.url=jdbc:mysql://localhost:3307/db_example
8  spring.datasource.driverClassName=com.mysql.cj.jdbc.Driver
9  spring.datasource.username=root
10 spring.datasource.password=123456
11
12 spring.jpa.hibernate.ddl-auto=update
13 spring.jpa.show-sql=true

```

- Bước 7: Chạy dự án
 - Run file **DemoApplication.java** (chứa **main**)
 - Mở Postman hoặc trình duyệt kiểm thử:

- GET `http://localhost:8080/api/users`
- POST `http://localhost:8080/api/users` (body JSON)

4. Tổng kết

- Cấu trúc dự án dịch vụ web thường gồm: **controller** → **service** → **repository** → **model**
- Có thể tạo nhanh dự án với **Spring Initializr**
- Sau khi triển khai, bạn có thể dùng **Postman** để test API

Danh sách các bài học

